LangGraph-M-H-Set2

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

01. Runtime dependencies

Answer B: Through runtime context, supplied for the invocation and accessed by nodes.

Runtime context is appropriate for invocation dependencies such as database clients. Nodes can receive Runtime and access runtime.context. The application supplies or recreates the dependency for the invocation; storing a live connection in checkpointed state would mix execution resources with business data.

Why the other choices are wrong

A. A live client is not ordinary serializable workflow data and should not be treated as a checkpointed result.

C. A message should contain model-facing content, not a live Python connection.

D. A node name identifies a computation; it is not dependency injection.

Remember: Keep invocation dependencies in runtime context and workflow facts in state.

Official references: [1]

02. Public output versus internal state

Answer D: Keep the internal state schema and define an output schema containing final_report.

Separate input, internal, and output schemas let a workflow retain working data while returning a narrower result. The output schema selects the public result fields. This is an API-shaping mechanism, not a security boundary: authorized checkpoint access or tracing can still expose internal state.

Why the other choices are wrong

A. A checkpointer persists state; it does not choose which fields invoke() returns.

B. Deleting fields the workflow needs prevents nodes from reliably exchanging that working data.

C. An underscore naming convention does not establish the graph output contract.

Remember: An output schema limits the returned result without deleting internal working state.

Official references: [5]

LangGraph-M-H-Set2

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

03. Adapting a subgraph interface

Answer A: Wrap the child invocation in a node that maps parent input and child output.

A wrapper node explicitly translates question into the child's query input, invokes the child, and returns the response as the parent's answer update. This keeps the child reusable and makes the boundary visible. Matching execution identifiers do not create a schema translation.

Why the other choices are wrong

B. Thread identity concerns persistence rather than renaming state fields.

C. Reducers combine updates to individual fields; a global renaming reducer is not this interface adapter.

D. Automatic semantic field-name matching is not part of direct subgraph integration.

Remember: Use a wrapper when parent and child schemas need translation.

Official references: [2]

04. Routing out of a subgraph

Answer C: Command(goto="finalize", graph=Command.PARENT)

Command.PARENT directs the command to the nearest parent graph, where finalize is defined. A destination without that graph scope refers to the current graph. This question concerns routing only; if the command also updates shared fields, the parent must have compatible state-update rules.

Why the other choices are wrong

A. Without parent scope, the destination is interpreted in the current child graph.

B. Fields named graph and next in a state update do not become routing instructions.

D. Putting graph in a Send payload does not select the parent graph namespace.

Remember: Use graph=Command.PARENT for an explicit handoff to the nearest parent graph.

Official references: [14]

LangGraph-M-H-Set2

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

05. Tenant-safe cache keys

Answer A: Include tenant and relevant corpus/version identity in the cache key.

The key must distinguish inputs that can legitimately produce different authorized results. Tenant and corpus identity affect retrieval, so ignoring them can reuse one tenant's cached evidence for another. Include the relevant identifiers in the node input and key. Cache isolation complements, rather than replaces, authorization.

Why the other choices are wrong

B. Trace labels do not change the explicitly defined cache key.

C. A longer TTL prolongs an incorrect cache entry; it does not isolate tenants.

D. Checkpointing records execution state and does not repair cache-key collisions.

Remember: A cache key must cover the factors that change the result and its permitted scope.

Official references: [1]

06. Visibility within a superstep

Answer B: 0, because node updates become visible in the next step.

LangGraph's bulk-synchronous execution separates node execution from applying their state updates. Both nodes read the step's input state, so B sees 0 even if A finishes first. If B must consume A's new value, make B depend on A in a later graph step.

Why the other choices are wrong

A. Finishing a node does not expose its returned channel update to peers in the same step.

C. With normal state updates and no external mutation, this is not a timing-dependent read.

D. Concurrent reads are allowed; conflicting writes are a separate concern.

Remember: A timing delay is not a data dependency; express dependencies in graph structure.

Official references: [3]

LangGraph-M-H-Set2

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

07. Reducer algebra

Answer D: It is not associative: the two displayed groupings produce 5 and 9.

For merge(a,b)=a-b, the left grouping is (10-3)-2=5 and the right grouping is 10-(3-2)=9. Different grouping therefore changes the result. A deterministic function is not necessarily associative. For additive contributions, addition has the required grouping property; business semantics must determine the reducer.

Why the other choices are wrong

A. A binary function need not be associative; the explicit calculation disproves that claim.

B. Saving intermediate values cannot change the algebraic property of subtraction.

C. Consistent types do not make different parenthesizations equivalent.

Remember: Check reducer algebra with concrete groupings, not just type signatures.

Official references: [4]

08. Business ordering after parallel work

Answer A: Include a section index in each result and sort during assembly.

Store ordering metadata with each section and apply the requested order explicitly after collecting the results. The scheduler's update order is not the report's business contract. Sorting by a stable unique section index separates parallel computation from presentation order and survives changes to node names or topology.

Why the other choices are wrong

B. Completion order reflects latency, not the requested document structure.

C. Node names are an implementation detail and do not enforce the business order.

D. Identical indices discard the information needed to order distinct sections.

Remember: Preserve a business ordering key and use it in the assembly step.

Official references: [15]

LangGraph-M-H-Set2

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

09. Manual edits still use reducers

Answer C: audit=["old", "corrected"], because the field reducer still applies.

update_state applies the same field-update rules used for node outputs. The list reducer concatenates the existing value with the supplied list. The call creates a new checkpoint; it does not mutate the historical checkpoint in place. A replacement requires an explicitly supported replacement mechanism or an appropriate schema design.

Why the other choices are wrong

A. A manual state update is not an implicit bypass of the field reducer.

B. There is no automatic clear step before this update.

D. Updates are allowed and create a new checkpoint rather than rewriting history.

Remember: A manual update is reducer-aware and produces a new checkpoint.

Official references: [6]

10. Attributing a state edit to a node

Answer D: Treat the update as output from draft and continue toward evaluate.

as_node identifies the node whose output the update is treated as representing. For this linear topology, the next scheduled work follows draft's outgoing edge to evaluate. The update does not execute the draft function. Resume using the configuration returned by update_state to continue from the new checkpoint.

Why the other choices are wrong

A. as_node affects execution attribution, not the name of a state field.

B. Attributing an update to a node does not call that node's function.

C. A human edit is not an automatic instruction to terminate.

Remember: as_node controls the execution position represented by a manual update.

Official references: [7]

LangGraph-M-H-Set2

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

11. Replaying from history

Answer B: It may differ because nodes after the selected checkpoint execute again.

Time travel reuses execution state up to the selected checkpoint and runs subsequent nodes again. The drafting node therefore calls the model again and can produce a different result. Replay is useful for exploring alternate trajectories, but it is not a guarantee of identical downstream external responses.

Why the other choices are wrong

A. A checkpoint preserves past state; it does not determine every future model response.

C. Keeping the same thread identity does not make a model response deterministic.

D. With no node cache, a downstream model call is performed again.

Remember: Distinguish replaying execution from merely viewing a saved final result.

Official references: [7]

12. Recovery with pending writes

Answer C: Reuse A's saved writes and retry the unfinished or failed work in B.

Pending-write recovery retains outputs from nodes that completed successfully in a failed superstep. Under the stated persistence conditions, A does not need to execute again; recovery continues the failed work. This differs from deliberately selecting an earlier checkpoint for a new replay, which can re-execute later nodes.

Why the other choices are wrong

A. The purpose of saved pending writes is to avoid repeating successful sibling work.

B. A failed branch still needs recovery; another branch succeeding does not satisfy it.

D. Saved successful output is retained for recovery, not permanently thrown away.

Remember: Recovery of a failed step can reuse successful sibling writes.

Official references: [6]

LangGraph-M-H-Set2

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

13. Selective retry policy

Answer A: Use bounded retries for ConnectionError and surface ValueError for correction.

Temporary connection failures can resolve on retry, so a bounded policy with backoff is appropriate. A malformed ID remains malformed on an identical retry and needs validation or input correction. The retry predicate should target known transient exceptions rather than treating all failures as equivalent.

Why the other choices are wrong

B. Silently returning empty evidence confuses a failure with a legitimate empty search result.

C. Read-only work can still waste resources indefinitely, and malformed input will not repair itself.

D. Delays alone do not classify errors or guarantee eventual termination.

Remember: Retry transient failures selectively; correct deterministic failures explicitly.

Official references: [13]

14. Migrating paused workflows

Answer B: Saved execution positions may still reference legal_review; preserve compatibility or migrate those runs.

Persisted executions can contain pending work referring to node names in the deployed graph. Removing or renaming a node needed by paused runs can break resumption. A deployment strategy should keep a compatible path or explicitly handle affected executions before retiring the old node.

Why the other choices are wrong

A. There is no general automatic semantic rename inference for persisted node references.

C. Node identity participates in execution, not just diagram labels.

D. A deployment change does not create a valid human approval decision.

Remember: Treat node identities needed by paused executions as compatibility commitments.

Official references: [1]

LangGraph-M-H-Set2

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

15. Interrupts and exception handling

Answer D: The handler can swallow the control-flow exception that signals the pause.

interrupt uses a special exception to transfer control to the graph runtime. A bare except catches it along with ordinary failures, so the runtime cannot handle the pause as intended. Keep the interrupt outside broad exception handling, or catch only specific operational exceptions around the operations that can raise them.

Why the other choices are wrong

A. A string can be a valid serializable interrupt payload.

B. Exception handling does not itself create the intended pause or a replacement checkpoint.

C. Calling interrupt inside a graph node is a supported usage pattern.

Remember: Do not swallow the exception that carries an interrupt to the runtime.

Official references: [8]

16. Ordering multiple interrupts

Answer C: Resume values are matched by call order within the task, so the changed order can misapply them.

Multiple interrupt responses within a task are matched by their position in the sequence of interrupt calls. Changing that sequence can make a stored name response satisfy the age prompt. Keep the call order stable across resumption, or redesign the workflow with separately managed steps and an explicit migration plan.

Why the other choices are wrong

A. Matching is not based on interpreting the prompt text.

B. The runtime supports multiple interrupts; it is not limited to persisting the last one.

D. An interrupt pauses the current execution rather than automatically creating a new thread.

Remember: Keep interrupt order stable for a task that may be resumed.

Official references: [8]

LangGraph-M-H-Set2

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

17. Correlating parallel tool results

Answer C: Each result's tool_call_id must match its corresponding requested call ID.

ToolMessages connect results to requested calls through tool_call_id. Two invocations of the same tool still represent separate requests, so one result must reference call_a and the other call_b. This matters when calls run concurrently or return in a different order from the model's requests.

Why the other choices are wrong

A. Inventing new tool names does not correctly identify which requested invocation produced a result.

B. Sharing a call ID loses the distinction between the two requested calls.

D. List position alone is not the protocol field used to correlate a tool response.

Remember: Correlate tool results by call ID, not tool name or completion order.

Official references: [9]

18. Preserving valid message history

Answer D: Preserve or remove the related tool-call/result exchange coherently.

Model-facing history must remain valid for the selected provider. Tool results depend on the assistant messages that requested them; retaining an orphaned result can break that structure. Trim at valid conversation boundaries, keeping required call/result pairs together or removing the exchange coherently.

Why the other choices are wrong

A. A random identifier cannot reconstruct the missing assistant tool call.

B. Changing roles blindly does not preserve the original conversation semantics or protocol.

C. Tool results alone do not form a valid replacement for the surrounding conversation.

Remember: Token-budget trimming must preserve valid conversation structure.

Official references: [10]

LangGraph-M-H-Set2

20 explained answers | Python Graph API | Official documentation checked October 28, 2026

19. Progress events without state pollution

Answer B: Emit through get_stream_writer() and consume the custom stream mode.

A stream writer can publish custom progress events from inside a running node. Consuming custom mode exposes those events to the caller without requiring them to become state fields. This provides incremental user feedback while the node continues its work; normal node returns remain the mechanism for state updates.

Why the other choices are wrong

A. Printing to process stdout does not automatically create a custom LangGraph stream event.

C. Waiting for the final output does not deliver live progress during the work.

D. A normal return ends the function; it is not an internal progress-emission mechanism.

Remember: Use custom stream events for application progress that need not be persisted in state.

Official references: [11]

20. Concurrency versus rate limits

Answer A: Concurrency limits simultaneous work; token-rate control needs separate budgeting or throttling.

Concurrency and rate measure different things. Four simultaneous tasks can make many short requests or consume large token counts. Configure token-aware admission or throttling, appropriate retry handling, and shared quota accounting where needed. A concurrency cap can help control load, but it is not a token-per-minute guarantee.

Why the other choices are wrong

B. Task count does not determine how many tokens a task consumes or how often tasks finish.

C. A concurrency cap can work correctly while a different resource limit is exceeded.

D. A graph thread identifier does not reset an external provider's account-level quota.

Remember: Limit simultaneous work and rate of consumption as separate constraints.

Official references: [12]

LangGraph-M-H-Set2

Source guide | Official LangChain / LangGraph Python documentation

References support the concepts behind the original practice questions. Code assumes standard imports and uses the Python StateGraph API. This set is independent practice material, not an official certification exam.

[1] Graph API: context, schemas, caching and migrations
https://docs.langchain.com/oss/python/langgraph/graph-api

[2] Subgraph interfaces
https://docs.langchain.com/oss/python/langgraph/use-subgraphs

[3] Pregel execution model
https://reference.langchain.com/python/langgraph/pregel/main/Pregel

[4] State and reducer reference
https://reference.langchain.com/python/langgraph/graph/state

[5] Graph API examples: parallelism and retry policies
https://docs.langchain.com/oss/python/langgraph/use-graph-api

[6] Checkpointers: updates and pending writes
https://docs.langchain.com/oss/python/langgraph/checkpointers

[7] Time travel and state attribution
https://docs.langchain.com/oss/python/langgraph/use-time-travel

[8] Interrupt rules
https://docs.langchain.com/oss/python/langgraph/interrupts

[9] Tool messages and identifiers
https://docs.langchain.com/oss/python/langchain/messages

[10] Memory: trimming and deleting messages
https://docs.langchain.com/oss/python/langgraph/add-memory

[11] Custom streaming
https://docs.langchain.com/oss/python/langgraph/streaming

[12] RunnableConfig: max_concurrency
https://reference.langchain.com/python/langchain-core/runnables/config/RunnableConfig

[13] Designing error handling
https://docs.langchain.com/oss/python/langgraph/thinking-in-langgraph

[14] Command routing reference
https://reference.langchain.com/python/langgraph/types/Command

[15] Runtime design and parallel execution
https://www.langchain.com/blog/building-langgraph